Entregable 3 — Resiliencia y observabilidad del Portal B2B

Aplicaciones y Sistemas Escalables · EAFIT · Equipo: Esteban Jacob Romero Rios · Juan José Rodríguez Restrepo · Santiago Sabogal Lozano **Entrega:** viernes 16 de octubre de 2026 · **Commit de evidencia:** ver `manifest.txt` de cada corrida

1. Resumen y alcance

Se interrogó la plataforma (3 instancias del monolito modular detrás de nginx, PostgreSQL 16, stub de crédito, RabbitMQ con outbox transaccional) con **8 fallas del catálogo, 2 por familia** (A latencia, B base de datos, C degradación, D disponibilidad), cada una **sin** y **con** mecanismos de resiliencia, **≥ 2 repeticiones válidas** por configuración, carga de 200 solicitudes/s y semilla 42. Cada falla tiene una ficha con predicción escrita el 2026-10-14 22:55, **antes de medir**; las fichas no se editaron. Las diferencias entre lo predicho y lo medido están en `Entrega 3/experimentos/<ID>_resultado.md` y se resumen aquí.

Resultado en una frase: de las 8 predicciones, 1 se cumplió casi exacta (A1), 4 se cumplieron en el mecanismo pero erraron números relevantes (A2, B1, C2, D1) y 3 fallaron de fondo (B2, C1, D2) —en cada una el fallo reveló un supuesto falso y produjo una corrección que se volvió a medir. Se descubrieron **dos defectos reales** (un 500 donde debía haber un 503 controlado, y un breaker que empeora el error) y **un hallazgo abierto** (50 mensajes retenidos en el consumidor).

Falla	Predicción	Veredicto	Lo más importante que se aprendió
A1 latencia fija en el externo	p95 global 770 ms sin / 300 con	**Cumplida** (765 / 268 ms)	Timeout 200 ms + 3 reintentos bajan el p95 global −65 % con 0,21 % de errores
A2 cola lognormal	p99 global 350 ms con M	**Parcial** (513 ms)	Los reintentos compran p99 (−50 %) pagando p95 (+14 %)
B1 PostgreSQL detenido	18–25 % de errores; recuperación 10–35 s	**Parcial** (7–8 %; 4–9 s)	La carga real cae durante la falla: los flujos son secuenciales
B2 bloqueo de la franja 91150	12 bloqueadas, hasta 45 s	**Falló** (0) → corregida	Una predicción puede fallar por el *estado* del sistema; `lock_timeout` baja el peor caso de 29 s a 2 s
C1 CPU saturada en app1	12 % del tráfico, p95 400 ms	**Falló** (26 %, 51 ms) → corregida	`least_conn` sí reparte; la falla era demasiado débil
C2 consumidor pausado	lag lineal ≈ 15 600; sin impacto HTTP	**Parcial**	Independencia confirmada; drenado más lento; 50 mensajes sin confirmar
D1 matar 1 de 3 instancias	2–8 × 502; reintegración ≤ 5 s	**Parcial**	2 × 502; la reintegración tardó 5–15 s; sondeo de nginx de 2 s
D2 5xx del crédito (50 %)	breaker no empeora el error	**Falló** (21 % vs 13 %) → corregida	Umbral del breaker = probabilidad de la falla ⇒ oscila; con 70 % el error baja a 1,7–2,7 %

2. Plataforma, método y línea base

Plataforma. Monolito modular (Órdenes, Proveedores, Catálogo, Logística; un esquema por módulo; fronteras verificadas en CI con import-linter), 3 réplicas stateless, nginx `least_conn` (`max_fails=3`, `fail_timeout=10s`, `proxy_connect_timeout=2s`), PostgreSQL 16 con 3,1 M de filas (Zipf, semilla 42), stub propio de crédito con fallas en caliente, RabbitMQ con transactional outbox y consumidor idempotente (`eventId`). Todo levanta con `make datos && make plataforma`.

SLO (RNF-10): p95 < 400 ms y errores < 0,5 % a 200 req/s. **Línea base oficial** (máquina del E2, `baseline/corridas/oficial-e3`, 300 s, semilla 42): p50 7,4 ms · p95 47,6 ms · p99 61,0 ms · 0 % de errores —

PASA. Los percentiles de este informe son p50/p95/p99 sobre `solicitudes.csv`, ventana de falla [91, 391) s del generador (60–360 s de la ficha); nunca promedios.

Método. Línea base → ficha con predicción numérica → inyección → medición → mecanismo → corrección → re-experimento. Cada corrida reinicia el estado (`make reset-estado`), registra un `manifest.txt` (commit, imágenes, configuración de Docker) y guarda `solicitudes.csv`, `eventos.log`, `recursos.csv` y la verificación de negocio (`outbox_total` = CONFIRMADAS, `reservada ≤ total`). **Perfil M (con resiliencia):** timeout por intento 200 ms (techo 5 s), 3 reintentos con backoff exponencial y *full jitter* y presupuesto 1,5 s (siempre con la misma `Idempotency-Key`), circuit breaker (ventana 10 s, ≥ 20 intentos, semiabierto con 5 pruebas), bulkhead de 20, degradación con `Retry-After`, `lock_timeout` 2 s y timeout de pool 2 s.

Validez de las corridas. Criterio: la mediana previa a la falla (inicio_s 31–90) debe estar cerca de 7 ms. El portátil tuvo períodos de *throttling* (sin enchufar, al sol); **13 corridas con mediana previa ≥ 12–15 ms se descartaron y se repitieron** (`Entrega 3/evidencia/LEEME_descartadas.md`), en lugar de reportar cifras contaminadas. Cuatro corridas válidas (D1 `con` r4, D2 `umbral70` r3 y r4, B2 `con-lock0` r6) están en el límite (mediana previa 10–15 ms) y se reportan con esa reserva; para ellas se usa el % de errores, que es robusto, y no las latencias absolutas.

3. Motor de fallas (las cinco condiciones)

1. **Sin redespliegue:** `faults/faults.yaml` se recarga en caliente; el controlador de fallas aplica y revierte las de infraestructura. 2. **Parametrizada:** probabilidad, latencia, jitter, distribución, duración, CPU, retención. 3. **Focalizable:** por endpoint, porcentaje de tráfico, contenedor, franja o instancia. 4. **Reversible en segundos:** *kill switch* (`enabled: false`) que también libera el bloqueo de B2 y restablece CPU de C1; el script de experimentos lo ejecuta con `trap` ante cualquier salida. 5. **Reproducible:** misma configuración + semilla 42 ⇒ misma secuencia de flujos (verificado: en B2 las solicitudes bloqueadas aparecen en los mismos instantes —15,5 s, 25,0 s, 27,0 s…— en todas las corridas).

4. Observabilidad

RED por endpoint y por instancia (`portal_http_*`, p50/p95/p99), USE por recurso (cAdvisor para CPU/memoria, pool de BD, colas de RabbitMQ), traza distribuida con `x-trace-id` y Jaeger, tablero único en Grafana (`uid portal-b2b`) con el p99 en tiempo real, y un muestreador propio por corrida (`recursos.csv`, `salud.csv`, `lag.csv`). **Límites honestos:** (a) varias métricas citadas en las fichas (`portal_bd_lock_timeouts_total`, `portal_degradaciones_total`) no existían; las cifras salen de `solicitudes.csv` y de contadores añadidos con las correcciones; (b) `rabbitmq_queue_messages_unacked` y las transiciones del breaker no se archivaron por corrida: se infieren de `lag.csv` y de las causas de error.

(Figuras: capturas de Grafana por experimento; las originales están en `Entrega 3/evidencia/<ID>/capturas/`.)

5. Resultados por experimento (predicho vs medido)

Cifras en la ventana de falla salvo indicación. “Sin” = valores por defecto; “con” = perfil M.

A1 · Latencia fija (30 % de las llamadas al externo, +500–1 100 ms). Predicción cumplida. p95 global: sin **765** ms (predicho 770) · con **268** (300). p99 global 1 066 / 552 (1 070 / 545). Errores: 0 % / **0,21 %** (0,22 %), todos 503 `TimeoutError` a 1,0 s; el breaker no abrió. Mecanismo: el 30 % de ≈ 27 % de las solicitudes es ≈ 8 % > 5 %, así que la cola lenta entra al p95. SLO: sin **NO**, con **SÍ**. *Evidencia:* `A1/{sin,con}_r{1,2}`.



Figura A1. Latencia p50/p95/p99 (arriba) y tasa de error global (abajo), sin resiliencia (izq., r1) y con perfil M (der., r1). Falla de 5 min en el centro de cada ventana.

A2 · Cola larga lognormal (σ = 1,5). Sin M casi exacto: p95 global 228 (260), p99 confirmar 1 171 (1 200), máximo 20 s. Con M el p99 de confirmar bajó **−50 % (1 180 → 585 ms)** y desaparecieron los rezagados de 20 s, pero **el p95 global subió 230 → 262 ms (+14 %)** y el p99 global (513) quedó por encima de lo predicho (350). Los reintentos intercambian p95 por p99: el 20 % de las llamadas con intento > 200 ms paga timeout + backoff. Errores 0,05 %. `con_r1` tuvo un episodio ajeno (522 errores) y se excluyó. *Evidencia:* `A2/{sin_r1,sin_r2,con_r2,con_r3}`.

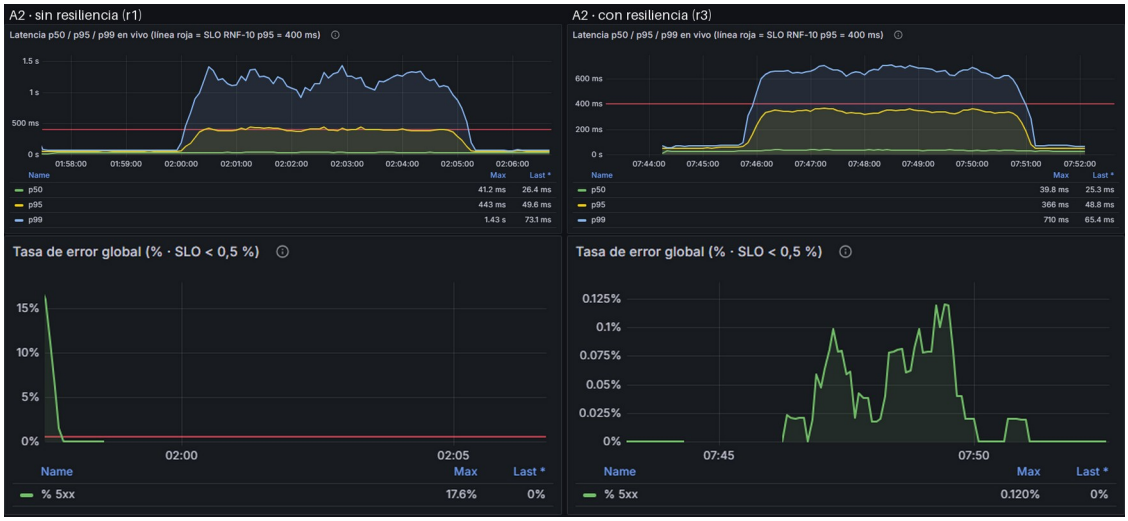


Figura A2. Sin resiliencia (r1) y con perfil M (r3): el p99 baja, pero el p95 sube; la tasa de error con M es ≤ 0,12 %. El pico de errores al inicio de la gráfica sin M es la cola de la corrida anterior, no parte de esta.

B1 · PostgreSQL detenido 45 s. Latencia de los errores como se predijo: **10,0 s** sin M y **2,0 s** con M (`BD_POOL_TIMEOUT_S=2`); p95 de la corrida 10,0 / 2,0 s. Dos números fallaron: (1) errores en la corrida **6,6–7,0 %** (sin) y **8,1–8,5 %** (con), no 18–25 %, porque un flujo es secuencial y un `crear` fallido cancela su `confirmar`: la carga real fue ≈ 104 req/s, no 200; (2) la recuperación fue de **4–9 s** tras `docker start`, no 10–35 s: el backoff del pool no se materializó. Con M se gana latencia del error (−80 %) a costa de más errores totales: es un intercambio espera ↔ error, no una mejora neta. Negocio: `outbox_total` = CONFIRMADAS en las 4 corridas (0 eventos perdidos), 0 violaciones de capacidad. Hallazgo abierto: `con_r3` mostró 109 respuestas `breaker_abierto` y 112 órdenes EN_PROCESO (causa no investigada). *Evidencia:* `B1/{sin_r1,sin_r2,con_r1,con_r3}`.

B2 · Bloqueo `FOR UPDATE` de la franja 91150. Original: **0** solicitudes bloqueadas, no las ≈ 12 predichas: la franja se llena a los 15,5 s del generador y desde entonces `UPDATE … WHERE reservada < total` no tiene fila candidata, así

que no espera el bloqueo (el caso borde del último cupo quedó protegido, pero la prueba no ejercitó el bloqueo). Variante `lock0` (bloqueo desde el arranque, 60 s): **sin M** 6 confirmaciones medidas (11 en total) esperaron hasta **28,8 s** (predicho 25–30); **con M** `lock_timeout` las cortó a **2,0 s** (−93 %), sin tocar p95/p99 globales. Esa re-ejecución destapó un defecto: las bloqueadas devolvían **HTTP 500** (`LockNotAvailable` sin mapear). Se corrigió en `servicio.py` (→ 503 `franja_bloqueada`, orden EN_PROCESO, clave liberada, `Retry-After`), con predicción fechada antes de re-ejecutar; las corridas r5/r6 confirmaron 11–12 × 503 y 0 × 500. Invariante `reservada ≤ total`: 0 violaciones en todas. *Evidencia:* `B2/{sin,con}_r{1,2}`, `B2/{sin,con}-lock0_r{3..6}`.

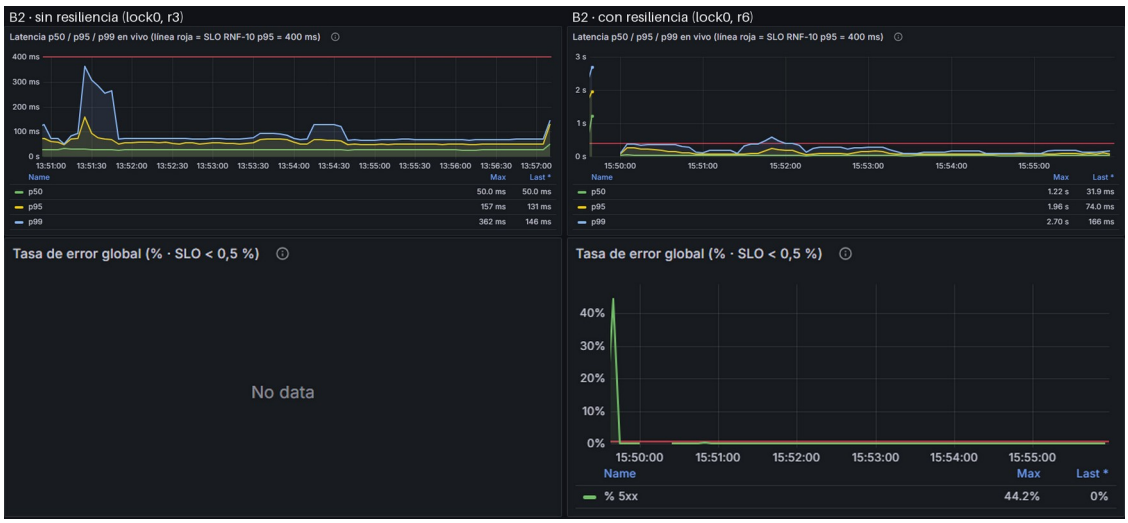


Figura B2. Bloqueo de la franja con `lock0`: sin M (r3) y con M (r6, con el mapeo a 503). El pico inicial de errores con M corresponde a la ventana del bloqueo (≈ 60 s).

C1 · CPU saturada en app1. Original (0,25 CPU): reparto **26/37/37 %** (predicho 12/44/44), p95 global **51–59 ms** (380–480): cada solicitud cuesta ≈ 4–5 ms de CPU y 0,25 CPU atiende ≈ 55 req/s, casi su cuota. Re-ejecución a **0,10 CPU** (predicción previa): reparto **11/44/44 %** ✓, p95 de app1 **394 ms** (predicho ≈ 300, rango 150–600) ✓, p95 global **98 ms** (≈ 100) ✓, 0 % de errores, SLO sí. `least_conn` sí desvía por carga efectiva. **El perfil M no aportó nada**: el `limit_req` de 250 req/s no se activa a 200 req/s; con M la cola de app1 fue peor (p95 544–601 vs 394 ms; "sin 0,10" tiene una sola repetición válida). La recuperación tardó ≈ 60 s, no ≤ 5 s. *Evidencia:* `C1/{sin,con}_r*`, `C1/{sin,con}-cpu010_r*`.

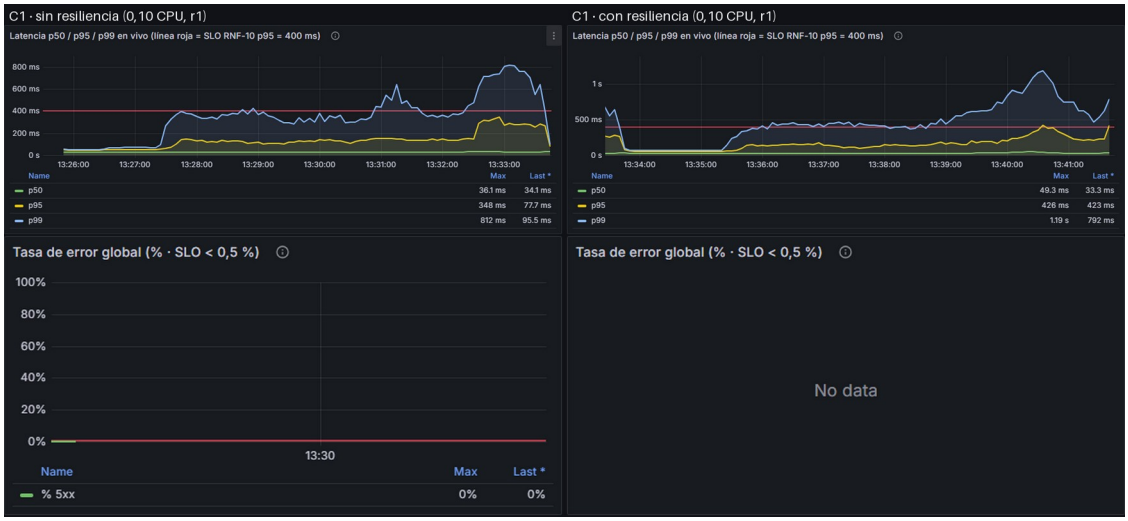


Figura C1. App1 limitada a 0,10 CPU: sin M (r1) y con M (r1). Con M la latencia crece a lo largo de la falla porque el reparto no corrige la falta de CPU.

C2 · Consumidor pausado 300 s. Independencia síncrono/asíncrono confirmada: p95/p99 HTTP **45–49 / 56–63 ms**, 0 % de errores. Cola de RabbitMQ creció linealmente hasta **15 285–15 405** (predicho ≈ 15 600), outbox ≤ 37, **0 eventos perdidos**

(`outbox_total` = CONFIRMADAS = 23 269). Falló el drenado: **74 s** (predicho 15–30 s) con p50 HTTP de 11–16 ms durante el catch-up. **Hallazgo abierto:** en 3 de 4 corridas quedan **exactamente 50 mensajes** (el `prefetch`) sin confirmar; en `con_r3` los eventos procesados son 23 219 de 23 269. No se perdió nada (siguen en la cola) pero la causa no está diagnosticada. *Evidencia:* `C2/{sin_r1,sin_r2,con_r1,con_r3}`.

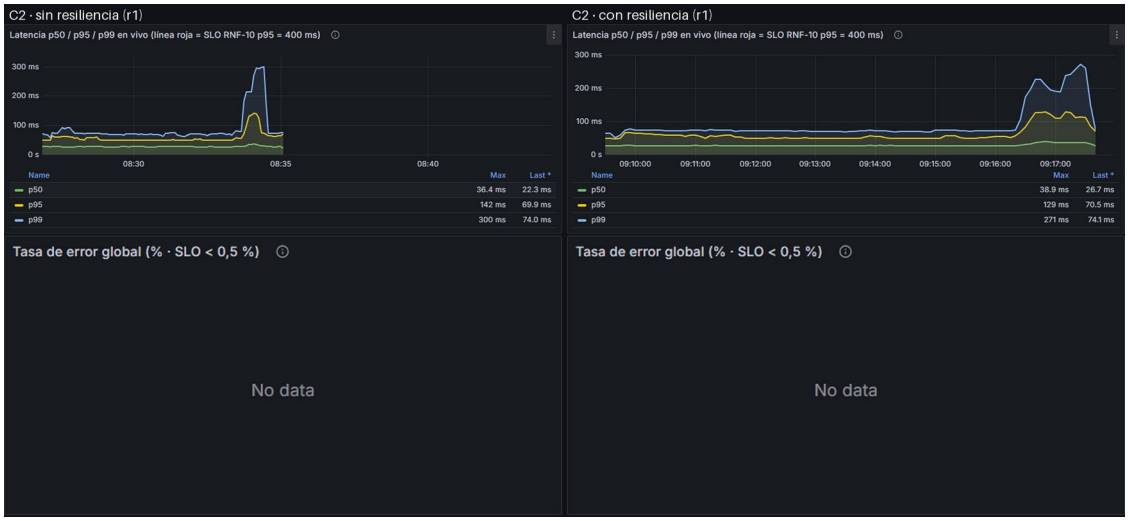


Figura C2. Consumidor pausado 300 s: la latencia HTTP no se altera ni sin ni con M (r1); 0 % de errores.

D1 · Matar app2 60 s. Reparto 50/0/50 % ✓, nginx dejó de enviarle tráfico desde el segundo siguiente a la muerte, **2 × 502** sin reintentos de cliente (predicho 2–8) y 0 con reintentos, errores 0,003 %, 0 duplicados, p95 **44–49 ms** (predicho 110–140: sobreestimado). No predicho: nginx sondea a la caída cada ≈ 13 s y cada sondeo cuesta ≈ **2,0 s** a 1–2 solicitudes (10–12 solicitudes > 1 s en 60 s); la reintegración tardó **4,6–14,9 s** (predicho ≤ 5). El reintento de cliente tiene un hueco: un 409 `confirmacion_en_curso` (clave dejada EN_CURSO por la instancia muerta) no es reintentable. *Evidencia:* `D1/{sin_r1,sin_r2,con_r1,con_r4}`.

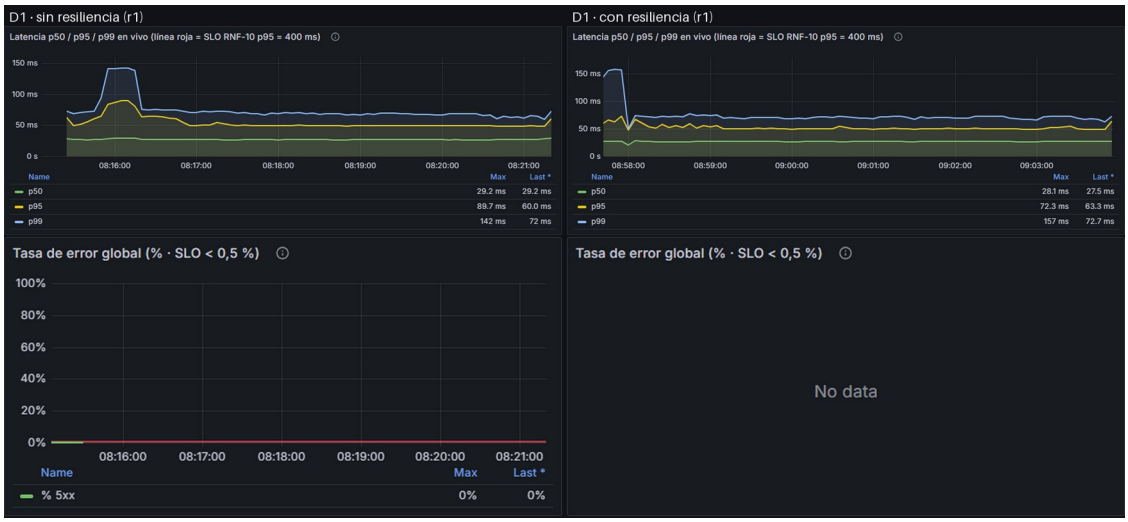


Figura D1. Instancia app2 detenida 60 s: sin M (r1) y con M (r1). La gráfica con M muestra el hueco de la instancia caída y ningún error 5xx.

D2 · 5xx del crédito al 50 %. Sin M exacto: **13,4 %** de errores (predicho 13,5). **Con M (umbral 50 %) falló: 21,0 %**, 12 700 de ellos por `breaker_abierto`: la falla devuelve 503 con probabilidad 50 %, igual al umbral, así que la tasa observada oscila alrededor y el breaker abre y cierra continuamente, rechazando incluso las llamadas que el stub habría atendido; el remedio **empeoró** el error de 13,4 a 21 %. Corrección (predicción previa): umbral **70 %**. Resultado: errores **1,7 % y 2,7 %** (predicho ≈ 1,7 %, 1–3 %), −80 a −87 % frente a "sin"; el residuo es $0{,}5^4$ = 6,25 % de las llamadas lógicas que agotan 4 intentos. En una de las dos corridas el breaker abrió igualmente (523 rechazos): subir el umbral reduce la oscilación pero no

la elimina con un host lento. RN-11 intacta: ninguna orden se aprobó sin crédito. SLO de errores sigue **NO** (esperado y declarado). *Evidencia:* `D2/{sin,con}_r{1,2}`, `D2/con-umbral70_r{3,4}`.

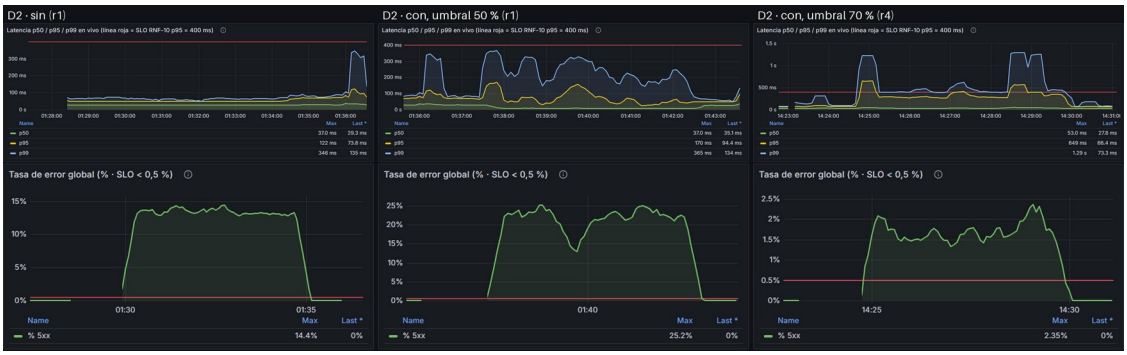


Figura D2. 5xx del crédito al 50 %: sin M (r1), con M y umbral 50 % (r1, 25 % de errores máx.) y con M y umbral 70 % (r4, ≈ 2,4 % máx.).

6. Qué se corrigió y qué se volvió a medir

Hallazgo	Causa raíz	Corrección	Re-medición
Breaker del 50 % empeoraba D2	Umbral = probabilidad de la falla	`BREAKER_UMBRAL_FALLOS_PCT=70`	21,0 % → 1,7–2,7 % de errores
B2 no ejercitaba el bloqueo	La franja se llena a los 15,5 s	Bloqueo desde el arranque (`lock0`)	0 → 11 solicitudes bloqueadas; 28,8 s → 2,0 s con `lock_timeout`
B2 devolvía 500 con `lock_timeout`	`LockNotAvailable` sin mapear	Mapeo a 503 `franja_bloqueada` en `servicio.py`	11 × 500 → 11 × 503; 0 × 500
C1 no estresaba la plataforma	Costo por solicitud subestimado (4–5 ms)	0,25 → 0,10 CPU	Reparto 26 → 11 %; p95 de app1 100 → 394 ms

Lo que más sorprendió: (i) la resiliencia no es monotónica: el mismo mecanismo (breaker, reintentos) mejoró A1, trade-off en A2 y empeoró D2; (ii) tres de ocho predicciones fallaron por supuestos sobre el *estado* o la *carga* del experimento, no sobre el sistema; (iii) el perfil M no sirve para C1 ni para B1 (intercambia espera por error).

7. Meta de mejora −10 %

Objetivos: 63 métricas del E2 con objetivo = 0,9 × valor del E2 (`baseline/objetivos.csv`); la línea base oficial del E3 ya cumple 8 sin tocar nada. Regla: una mejora menor que el rango entre corridas **no es mejora** (`baseline/palancas.md` §1).

Palancas de consulta (P1–P3), A/B alternado antes/después en la misma base, N = 5:

Palanca	Cambio	Efecto medido	Veredicto
P1 índice cubriente Q4 (`(proveedor_id, fecha_emision DESC, id DESC) INCLUDE (estado, fecha_compromiso, fecha_entrega)`)	Bitmap Heap Scan → **Index Only Scan, 0 Heap Fetches**; ejecución en servidor 1,93 → 1,46 ms (−25 %)	p95 Q4 caliente 14,9 → 16,1 ms (rango ±10 ms)	**Dentro del ruido** de punta a punta
P2 `jit = off`	No se verificó que el plan usara JIT	Sin mejora; algunas métricas empeoran dentro del ruido	Descartada
P3 `random_page_cost = 1,1`	—	Mejoras y empeoramientos sin patrón; Q5 OFFSET 5 000 p95 4,0 → 4,2 ms	Dentro del ruido; costo: afecta todos los planes

Conclusión honesta: P1 mejora el plan pero el costo de punta a punta de Q4 lo domina otra cosa (lo más probable, el cliente procesando ≈ 4 000 filas; no se midió por separado), y las consultas de 0,5–1 ms (Q1, Q3, Q5 página 1) están por debajo del ruido del cliente: **no aplicable**.

Palanca P8 (E4, ruta transaccional): validar el catálogo con una sola consulta por orden y cargar las líneas con `executemany` (pipeline). Cambia el código del benchmark, no el motor: se declara como mejora de la ruta. Regla de equivalencia: mismas órdenes aceptadas y mismas filas insertadas. **Resultado P8.** A/B alternado sobre las primeras 10 000 órdenes (3 repeticiones, control y `lote` intercalados): mediana **134,8 s → 68,3 s (−49,3 %)**; el rango del control es 18,3 %, así que el efecto está ~3 veces por encima del ruido. Equivalencia cumplida: 7 783 órdenes aceptadas y 58 589 filas insertadas en ambas variantes. E4 completo (50 000 órdenes): **298,9 s** frente a 486,42 s del E2 (**−38,5 %**, objetivo 437,78 s) y frente a 437,6 s de la línea base oficial del E3 (−31,7 %); 29 235 aceptadas y 219 720 filas, idéntico a la línea base. Rendimiento: 735 filas/s (objetivo ≥ 502). Matiz: la línea base oficial del E3 ya había quedado en 437,6 s (−10,0 % sin tocar nada, una sola corrida), por lo que ese cumplimiento es marginal y no se atribuye a ninguna palanca; el efecto de P8 se apoya en el A/B alternado y en que las demás estrategias (E1–E3) salieron *más lentas* en esa misma sesión (E2 11,2 s frente a 10,4 s; E3 56,1 s frente a 43,4 s), es decir, el equipo no estaba más rápido. Limitación: P8 modifica el código del benchmark (la ruta transaccional), no el motor de la base de datos, y el E4 completo con `lote` se midió una vez.

8. Conclusiones, limitaciones y trabajo pendiente

1. **Con el perfil M la plataforma cumple el SLO en 6 de las 8 fallas** (A1, A2, B2, C1, C2, D1) y lo incumple por diseño en B1 (sin BD no hay confirmación posible) y D2 (50 % de 503 del externo deja ≈ 1,7 % de errores residuales, sobre el 0,5 % del presupuesto).
2. **Nunca se aprobó una orden sin crédito, nunca se perdió un evento y la invariante `reservada ≤ total` no se violó** en ninguna de las corridas válidas (`outbox_total` = CONFIRMADAS; 0 violaciones).
3. **Defectos abiertos:** (a) 50 mensajes sin confirmar tras reanudar el consumidor (C2); (b) 109 `breaker_abierto` y 112 EN_PROCESO en B1 `con_r3`; (c) el 409 no reintentable tras matar una instancia (D1); (d) el breaker con umbral 70 % puede abrirse igualmente con un host lento.
4. **Limitaciones de la medición:** el reloj del contenedor avanza ≈ 4 % más rápido que el de pared (la falla puede desfasarse unos segundos de la ficha); corridas descartadas por *throttling* del equipo (13); cuatro corridas válidas con reserva; "sin 0,10 CPU" con una sola repetición; el rate limit se implementó en nginx porque `RATE_LIMIT_*` no estaba conectado en la app; el breaker no se probó con 100 % de fallas del crédito; los episodios de la plataforma ajenos a la falla (A2 `con_r1`, B2 `con-lock0_r5`) se conservan como evidencia pero no entran en las cifras.
5. **Qué haríamos con más tiempo:** diagnosticar el lote retenido del consumidor; archivar las transiciones del breaker y los unacked por corrida; bajar `proxy_connect_timeout` en nginx; probar el breaker con una caída total del crédito; repetir cada experimento en una máquina sin *throttling*.

Reproducir: `make datos && make plataforma && make baseline`; un experimento: `bash faults/experimento.sh <ID> <sin|con|con-VARIANTE> <rep>`; todo el catálogo: `make cola`. Evidencia: `Entrega 3/evidencia/<ID>/<config>_r<n>/` y `Entrega 3/experimentos/`.